

Cross-Chain Smart-Contract Verification

Eight Platforms, One Proof Kernel

Mathieu Dutour Sikirić

Using Verus to verify fungible-token logic across
CosmWasm · NEAR · IC · Gear · ink! · MultiversX · Solana · Linera

The bug we want to make impossible

A fungible-token contract holds a balance map and a `total_supply` field.

The invariant every honest implementation should satisfy:

$$\sum_{a \in \text{accounts}} \text{balance}(a) = \text{total_supply}$$

The bug class:

- **Compound** (Sep 2021): MASTER inflation, \$80M.
- **USDC** (theoretical via faulty `_mint`): unbounded creation.

- **TheDAO** (Jun 2016): not conservation per se, but a different invariant violated.

A single overflow on `to_balance + amount` and the sum no longer matches the supply. Tests with concrete inputs cannot prove the invariant holds for **all** inputs.

Tests check examples. Proofs check classes.

Testing

- Pick inputs (from, to, amount).
- Run the contract.
- Assert post-state.
- Catches the bugs you thought of.

Verification

- Assert a property for **all** inputs.
- Verifier searches for a counter-model.
- Either a proof, or a witness.
- Catches the bugs you didn't.

We want this to be a **theorem**:

For any pre-state s , any (from, to, amount), if transfer returns Ok then $\text{sum}(\text{balances})$ in the post-state equals $\text{sum}(\text{balances})$ in s .

Verus in one slide

Refinement-typed Rust. You annotate the same Rust that compiles to wasm; the verifier checks the annotations match the body. No separate spec language.

```
pub fn transfer_balances(
    from_balance: u128, to_balance: u128, amount: u128,
) -> (r: Result<(u128, u128), &'static str>)
    ensures match r {
        Ok((from_next, to_next)) =>
            from_next + to_next == from_balance + to_balance //
conservation
            && from_next == from_balance - amount
            && to_next == to_balance + amount,
        Err(_) =>
```

```

        from_balance < amount || to_balance + amount > u128::MAX,
    },
{
    match from_balance.checked_sub(amount) {
        None => Err("insufficient balance"),
        Some(from_next) => match to_balance.checked_add(amount) {
            None => Err("balance overflow"),
            Some(to_next) => Ok((from_next, to_next)),
        },
    }
}

```

Verus discharges this to Z3. 1 second on a laptop. The function **cannot** be implemented incorrectly and still typecheck.

The question this work answers

Most verification work targets one platform. EVM/Solidity is the obvious one (K-framework, Certora, Slither).

But there are **eight major non-EVM smart-contract platforms**:

**CosmWasm · NEAR · Internet Computer · Gear · ink! ·
MultiversX · Solana · Linera**

Each has its own SDK, storage primitive, caller-resolution mechanism, error idiom. **Naïve verification = one project per chain.**

What is the right shape of a verification project so that the proof is portable?

Eight chains, eight runtime shapes

Chain	Caller	Storage primitive	Money type
CosmWasm	MessageInfo.sender	Map<K,V> via cw-storage-plus	Uint128
NEAR	env::predecessor_account_id()	LookupMap<K, V, Identity>	u128
IC	ic_cdk::api::caller()	thread_local! BTreeMap	u128
Gear	msg::source()	static mut + HashMap	u128
ink!	Self::env().caller()	ink::storage::Mapping	Balance (u128)
MultiversX	self.blockchain().get_caller()	SingleValueMapper	BigUint (unbounded)
Solana	AccountInfo.is_signer	account-data buffers (Borsh)	u128
Linera	runtime.authenticated_signer	SyncMapView<K,V>	Amount (newtype u128)

The common shape is always the same: **(caller, recipient, amount) → two balance reads → two checked-arithmetic updates → two writes**. The question is how much of that can be one proof.

The three-layer architecture

Layer 3 — chain-specific entry points

`#[entry_point] / #[update] / #[ink(message)] / extern "C" fn handle()...`

1–10 lines of **unverified** glue per entry. Forwards to:



Layer 2 — verified helpers (inside `verus!{}`)

`verified_transfer(storage, sender, receiver, amount) -> Result<...>`

Reads, runs arithmetic, writes. ensures describes the abstract storage delta.



Layer 1 — chain-agnostic core (verus_fungible_core crate)

transfer_balances : u128, u128, u128 → Result<...> + conservation lemmas
on State<A>.

Byte-identical across all eight chains.

Trust surface — <chain>_axioms.rs: external_type_specification for SDK types,
point-op storage axioms, caller ghost.

Conservation, stated once for all chains

Inside the shared core crate, generic over the chain's account-id type A:

```
pub struct State<A> {
    pub total_supply: nat,
    pub balances: Map<A, nat>,          // unbounded-precision in the spec
}

impl<A> State<A> {
    pub open spec fn invariant(self) -> bool {
        self.balances.dom().finite()
        && sum_balances(self.balances) == self.total_supply
    }
}

pub proof fn lemma_transfer_preserves_invariant<A>(
    s: State<A>, from: A, to: A, amount: nat,
)
    requires s.invariant(), from != to,
```

```
        s.balances.dom().contains(from), s.balances.dom().contains(to),  
        s.balances[from] >= amount,  
    ensures state_after_transfer(s, from, to, amount).invariant(),  
{ /* 15-line proof by induction on the map's domain */ }
```

Proven once, reused eight times. Bridge to u128 is a separate refinement lemma, also in the shared crate.

Case 1: CosmWasm — what “done” looks like

The whole cw20 surface — transfer, approve, transfer_from, mint, burn, increase_allowance, decrease_allowance, update_minter — is routed through verified helpers.

```
#[entry_point]
pub fn execute(deps: DepsMut, _env: Env, info: MessageInfo, msg: ExecuteMsg)
    -> Result<Response, ContractError>
{
    let mut store_ref = StoreRef(deps.storage);
    match msg {
        ExecuteMsg::Transfer { recipient, amount } => {
            let to = deps.api.addr_validate(&recipient)?;
            verified_transfer(&mut store_ref, &info.sender, &to, amount.u128())
                .map_err(map_transfer_error)?;
            Ok(Response::new().add_attribute("action", "transfer"))
        }
        // ... 7 more branches, each a forwarder ...
    }
}
```

```
}  
}
```

11 verified obligations in CosmWasm + **12 in the shared core** = the substantive arithmetic, storage refinement, authorization, and state-level connection. **24 unit tests** still pass.

Unverified glue: the `entry_point` macro expansion and the `addr_validate` SDK call (5 lines per branch).

Case 2: MultiversX — when unbounded is better

MultiversX uses `BigUint<M>`: an **unbounded** arbitrary-precision integer (managed by the runtime). Our shared core uses `u128`. They don't compose — naïve conversion silently truncates.

We axiomatized `BigUint` directly, mapping it to spec-level `nat`:

```
pub uninterp spec fn bigint_val<M: ManagedTypeApi>(x: &BigUint<M>) -> nat;
```

```
pub fn verified_transfer_big<M: ManagedTypeApi>(
  from_balance: BigUint<M>, to_balance: BigUint<M>, amount: &BigUint<M>,
) -> Result<(BigUint<M>, BigUint<M>), ()>
  ensures Ok((from_next, to_next)) =>
    bigint_val(&from_next) + bigint_val(&to_next)
      == bigint_val(&from_balance) + bigint_val(&to_balance);
```

The surprise: no overflow precondition. `BigUint` is unbounded, so addition is unconditional. Compare to the `u128` core, which must require `to_balance + amount <= u128::MAX`. **BigUint**

maps to nat more naturally than u128 does — provided you can axiomatize the trait machinery.

The obstacle, and the discovery

BigUint<M: ManagedTypeApi> requires Verus to know ManagedTypeApi, which has **five super-traits**:

```
ManagedTypeApi: HandleTypeInfo + StaticVarApi + ErrorApi + Clone + 'static
```

Three forms I tried failed:

1. `type ExternalTraitSpecificationFor: ManagedTypeApi + HandleTypeInfo + ...;`
→ **only one bound allowed**.
2. `where Self::ExternalTraitSpecificationFor: HandleTypeInfo, ...` on the associated type
→ **bounds don't match exactly**.
3. Proxy-trait inheritance `pub trait ExManagedTypeApi: ExHandleTypeInfo + ... → Verus`
sees the inherited bounds but **doesn't equate proxy names with external trait names**.

The form that **does** work — put the external traits in the proxy's super-trait list directly:

```
#[verifier::external_trait_specification]  
pub trait ExManagedTypeApi:
```

```
HandleTypeInfo + StaticVarApi + ErrorApi + Clone + 'static
{
  type ExternalTraitSpecificationFor: ManagedTypeApi;
}
```

Same fix unblocked Linera's SyncMapView<C, K, V>. Pattern is now in the project's DESIGN.md.

Case 3: NEAR — a wall we can't see around

`near_sdk::store::LookupMap<K, V, H: ToKey>` is NEAR's persistent storage. `ToKey` has a **private** super-trait:

```
// in near_sdk
mod private { pub trait Sealed {} }
pub trait ToKey: private::Sealed { /* ... */ }
```

Verus needs to declare `ToKey` as external. But `external_trait_specification` requires listing super-traits exactly — and `Sealed`'s path is **not accessible** outside `near_sdk`. We can't write `pub trait ExToKey: Sealed`.

Workaround: wrap `LookupMap<K, V, Identity>` in our own `AxLookupMap<K, V>` newtype, specialized to the default hasher. No generic-hasher support.

Lesson: SDK encapsulation can be a hard wall for whole-program verification. Tools can't see through privacy. This is a real argument for SDK authors to consider verifier-friendliness as a design constraint.

What's in the tree

Chain	Verified	Tests	Notes
verus_fungible_core (shared)	12	—	arithmetic + State + 3 refinement lemmas
CosmWasm	11	24	full cw20 surface, state-level refinement
NEAR	3	6	verified transfer; drift in core.rs restored by shared crate
IC	8	4	full cw20 surface, no state-level refinement yet
Gear	2	6	handle() dispatch routes through verified kernel
ink!	2	10	arithmetic verified; Mapping storage gap remains
MultiversX	1	6	verified_transfer_big via trait-cascade pattern
Solana	8	15	process_instruction dispatches to verified instruction
Liner	8	—	credit / debit via SyncMapView axiomatization

All chains: 0 errors. Trust surface per chain: 30 lines of axioms + 5 lines of glue per entry point. Shared crate eliminates 440 lines of previously duplicated `core.rs` content.

Where this sits in the verification landscape

Tool	Target	Style
K framework	EVM bytecode	operational semantics, symbolic execution
Certora	Solidity / Yul	CVL spec language, SMT
Slither / Mythril	Solidity	static analysis, bug-finding
Halmos / Foundry-invariant	Solidity	bounded symbolic / property testing
Verus (this work)	Rust → wasm	refinement on production code

The differentiator: Verus annotations live on the same Rust code that compiles to the deployed wasm. No separate spec artifact to drift. Any chain whose contract language is Rust (or compiles via Rust) is in scope — including most non-EVM chains.

The tradeoff: each chain's host semantics is an axiom you write by hand. 30 lines per chain, trusted, kept small.

Open problems

1. Cross-contract dispatch (reentrancy).

Every chain has it: CosmWasm SubMsg, NEAR Promise, Solana CPI, IC inter-canister, Linera cross-microchain messages. Today's verification stops at one contract's boundary. Cleanly modelling “between our return and the reply, an attacker can re-enter” requires an uninterpreted-relation framing. 2–4 weeks per chain. **This is where production-cw20 bugs concentrate.**

2. State-level connection on intermediate chains.

IC, Linera have verified kernels but no nat_balances-bridged conservation theorem at the entry point. Mechanical port from CosmWasm.

3. The Solana framing layer.

Write A doesn't change Account B's view, but our pure-function ghost model doesn't say that.
Needs explicit tracked ghost state. 1 week.

4. **MultiversX** `SingleValueMapper` and **Linera** `verified_approve`: pieces left when

`OwnerSpender::new` panics on `owner == spender`. Both 2-4 days of focused work each.

Three lessons

1. The hardest part is not writing proofs; it's making proofs portable.

The conservation theorem was straightforward. Connecting it to eight different SDKs took 80% of the effort.

2. SDK privacy is the verifier's wall.

The single biggest blocker (NEAR's `Sealed`) is a privacy decision in an SDK. Verifier-friendliness should be a design property of platforms.

3. “Same proof, different host” is the right architectural goal.

Three layers — shared core, chain axioms, chain glue — is the minimum that lets the same lemma about conservation cover eight runtimes. Anything less and you redo the proof. Anything more and you can't keep up with SDK churn.

**Eight chains. One conservation theorem.
Zero verification errors.**

The bug class that lives in fungible-token contracts since 2016
is now structurally impossible across this proof surface.

Questions?